

Indian Institute of Technology
Department of Computer Science

CS202: Software Tools and Technologies Assessment 3

Jaskirat Singh Maskeen and Aarsh Wankar
(23110146, 23110003)

Instructor: Prof. Shouvick Mondal

November 8, 2025

Contents

9	Development of C# Console Applications	1
9.1	Introduction	1
9.2	Setup and Tools	1
9.3	Methodology, Execution and Results	3
9.3.1	Understanding Basic Syntax and Control Structures	3
9.3.2	Usage of loops and functions	5
9.3.3	Usage of single and multi-dimensional arrays	6
9.4	Output Reasoning Analyses	9
9.4.1	Output Reasoning (Level 0)	9
9.4.2	Output Reasoning (Level 1)	10
9.4.3	Output Reasoning (Level 2)	12
9.5	Conclusion	14
	References	14
10	Object Lifecycle, Inheritance, and Event-Driven Programming in C#	15
10.1	Introduction	15
10.2	Setup and Tools	15
10.3	Methodology, Execution and Results	15
10.3.1	Constructors and Data Control	15
10.3.2	Inheritance and Method Overriding	17
10.4	Output Reasoning Analyses	19
10.4.1	Output Reasoning (Level 0)	19
10.4.2	Output Reasoning (Level 1)	21
10.4.3	Output Reasoning (Level 2)	25
10.5	Conclusion	28
	References	28

Lab 9

Development of C# Console Applications

[Github Repository for Lab 9](#)

9.1 Introduction

In this lab, we will set up and use Visual Studio for .NET development, write and execute basic C# console applications, and implement fundamental programming constructs such as loops, conditionals, and functions. Additionally, we will apply object-oriented programming concepts in C# to develop structured and efficient programs.

9.2 Setup and Tools

We use Visual Studio 2022 Community Version for .NET development and writing C# code along with .NET 8.0 (LTS) SDK. We use Windows 11 as our operating system. To setup the .NET Development Environment, we first download Visual Studio from their official website, and install it with .NET support enabled. Next, we create a new project and name it STT_LAB_9. The screenshots are shown below:

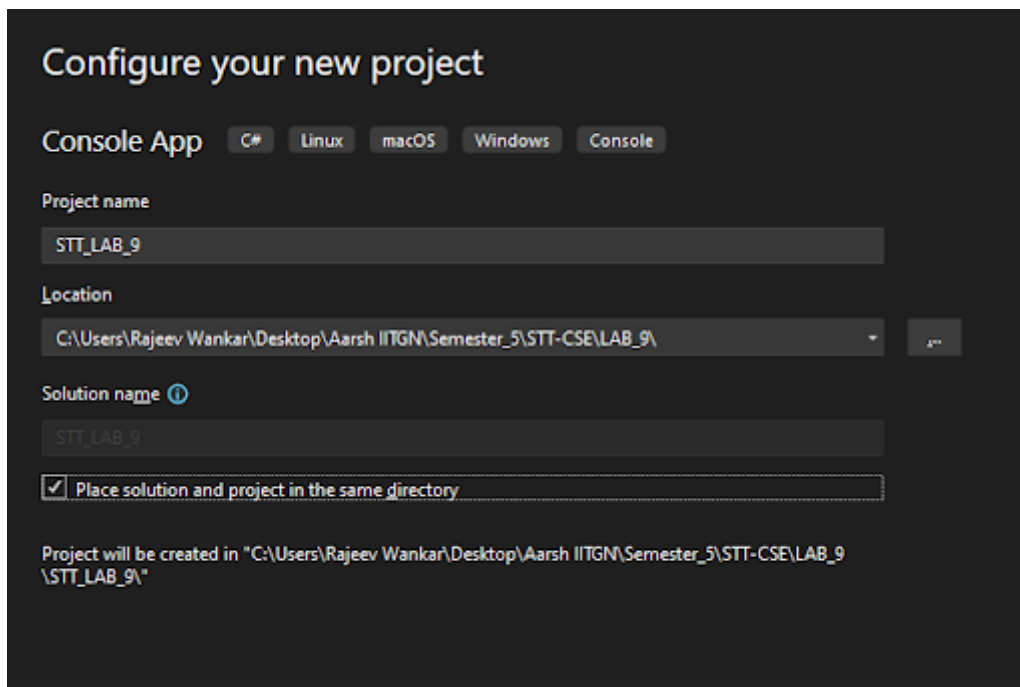


Figure 9.1: Creating a new project

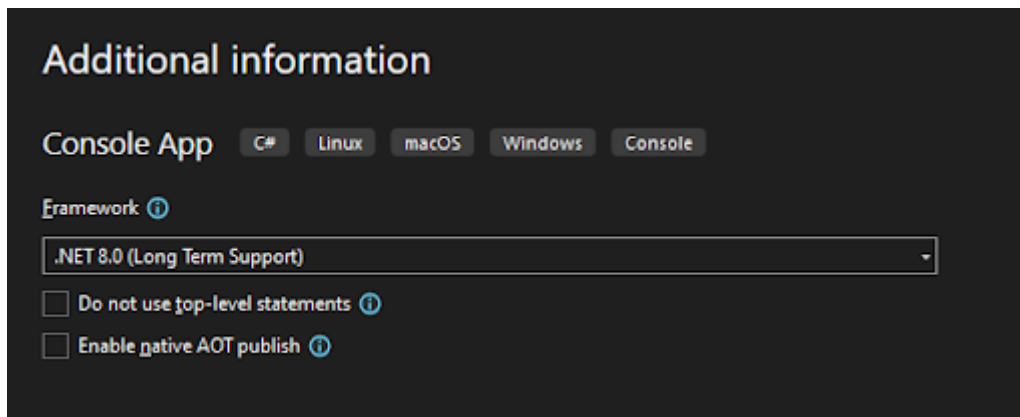


Figure 9.2: Selecting the .NET version

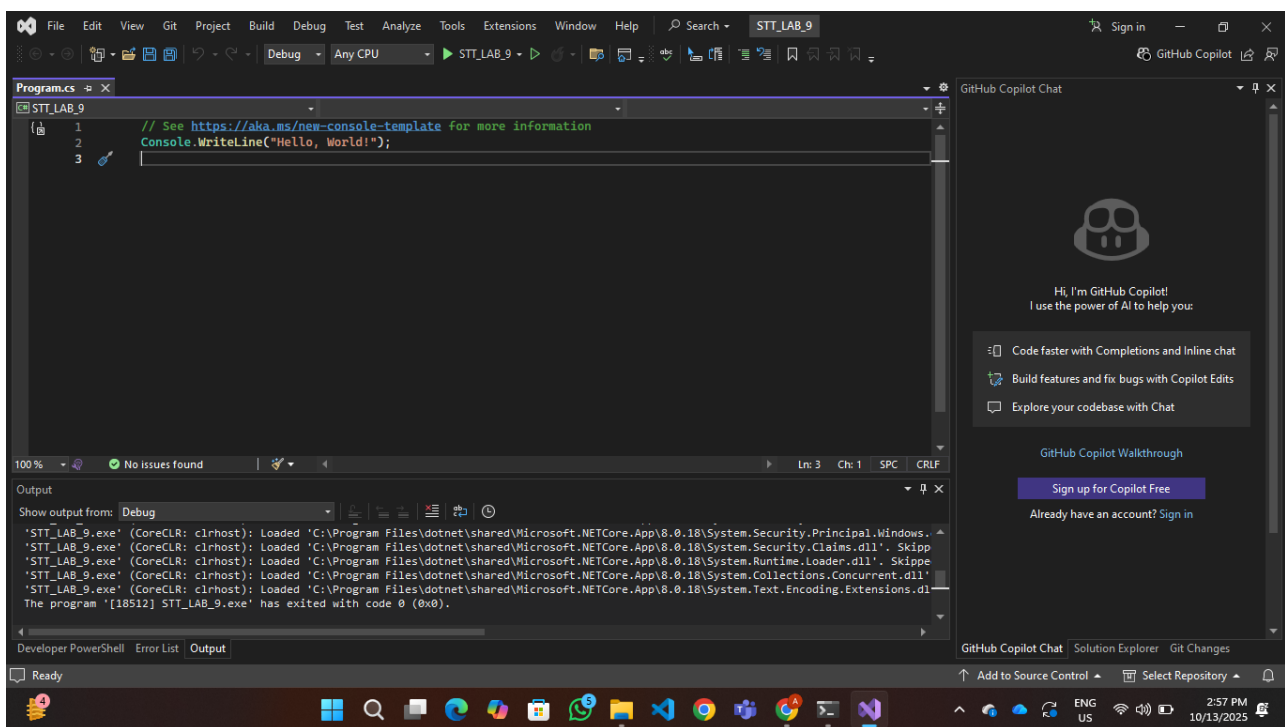


Figure 9.3: Visual Studio starting window

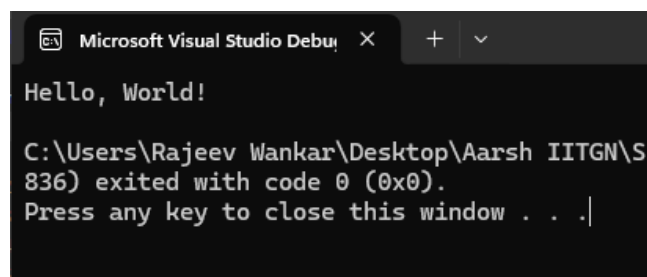


Figure 9.4: Output of the Hello world program

9.3 Methodology, Execution and Results

9.3.1 Understanding Basic Syntax and Control Structures

We write an object-oriented C# program that accepts user input for two numbers, performs addition, subtraction, multiplication, and division, and uses if-else conditions to determine whether the sum is even or odd. We will then display all results using the `Console.WriteLine()` method. The code is given below:

```
1  using System;
2
3  namespace Program
4  {
5      class Calculator
6      {
7          public int A { get; }
8          public int B { get; }
9
10         public Calculator(int a, int b)
11         {
12             A = a;
13             B = b;
14         }
15
16         public int Add() => A + B;
17         public int Subtract() => A - B;
18         public int Multiply() => A * B;
19         public double Divide()
20         {
21             if (B == 0) throw new DivideByZeroException("Cannot divide by
zero.");
22             return (double)A / B;
23         }
24     }
25
26     class Program
27     {
28         static int ReadInt(string prompt)
29         {
30             while (true)
31             {
32                 Console.Write(prompt);
33                 string? input = Console.ReadLine();
34                 if (int.TryParse(input, out int value))
35                     return value;
36
37                 Console.WriteLine("Invalid input. Please enter a valid
integer.");
38             }
39         }
40
41         static void Main(string[] args)
42         {
43             Console.WriteLine("Simple OOP Calculator");
44
45             int first = ReadInt("Enter the first integer: ");
46             int second = ReadInt("Enter the second integer: ");
47
48             var calc = new Calculator(first, second);
49
50             int sum = calc.Add();
51             int difference = calc.Subtract();
```

```
52         int product = calc.Multiply();
53
54         Console.WriteLine();
55         Console.WriteLine($"Addition:           {first} + {second} = {sum}"
56     );
57         Console.WriteLine($"Subtraction:        {first} - {second} = {
58 difference}");
59         Console.WriteLine($"Multiplication:    {first} * {second} = {
60 product}");
61
62         if (second == 0)
63         {
64             Console.WriteLine($"Division:           {first} / {second} =
65 Undefined (division by zero)");
66         }
67         else
68         {
69             double quotient = calc.Divide();
70             Console.WriteLine($"Division:           {first} / {second} = {
71 quotient}");
72         }
73
74         if (sum % 2 == 0)
75         {
76             Console.WriteLine($"The sum ({sum}) is even.");
77         }
78         else
79         {
80             Console.WriteLine($"The sum ({sum}) is odd.");
81         }
82     }
83 }
```

The output is shown below:

```
Simple OOP Calculator
Enter the first integer: 10
Enter the second integer: 20

Addition:           10 + 20 = 30
Subtraction:        10 - 20 = -10
Multiplication:     10 * 20 = 200
Division:           10 / 20 = 0.5
The sum (30) is even.

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\STT-CS
324) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Figure 9.5: Output of the calculator program

9.3.2 Usage of loops and functions

We now write a program for demonstrating the use of loops and functions in C# within an object-oriented structure, where the program prints numbers using for and foreach loops, repeatedly accepts user input until "exit" is entered using a do-while loop, and calculates the factorial of a user-provided number through a dedicated function.

The code is shown below:

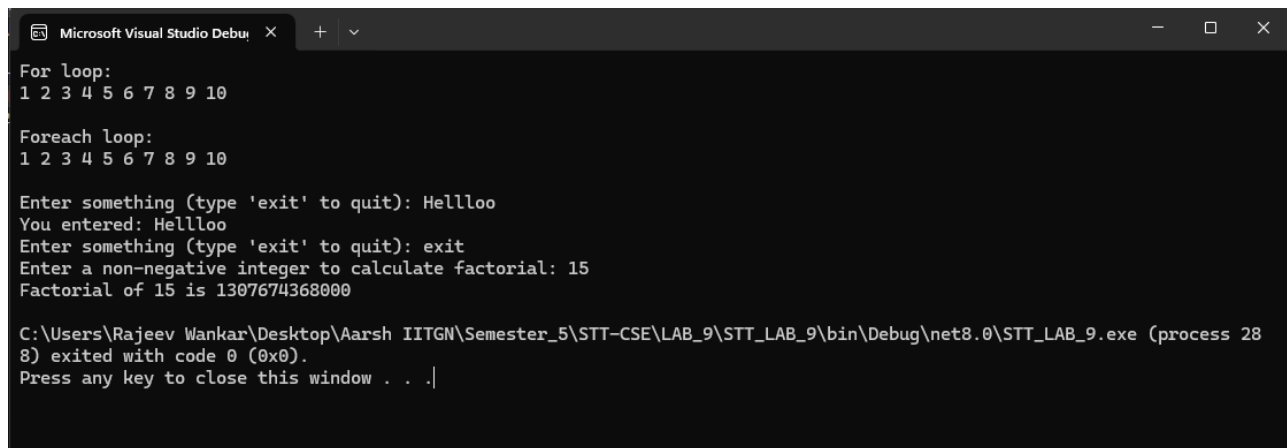
```
1  using System;
2  using System.Collections.Generic;
3
4  namespace LoopFunctionDemo
5  {
6      class Program
7      {
8          static long Factorial(int n)
9          {
10             long result = 1;
11             for (int i = 2; i <= n; i++)
12             {
13                 result *= i;
14             }
15             return result;
16         }
17
18         static void Main(string[] args)
19         {
20             Console.WriteLine("For loop:");
21             for (int i = 1; i <= 10; i++)
22             {
23                 Console.Write(i + " ");
24             }
25             Console.WriteLine("\n");
26
27             Console.WriteLine("Foreach loop:");
28             List<int> numbers = new List<int>();
29             for (int i = 1; i <= 10; i++) numbers.Add(i);
30
31             foreach (int num in numbers)
32             {
33                 Console.Write(num + " ");
34             }
35             Console.WriteLine("\n");
36
37             string input;
38             do
39             {
40                 Console.Write("Enter something (type 'exit' to quit): ");
41                 input = Console.ReadLine();
42                 if (input.ToLower() != "exit")
43                     Console.WriteLine("You entered: " + input);
44             } while (input.ToLower() != "exit");
45
46             int number;
47             while (true)
48             {
49                 Console.Write("Enter a non-negative integer to calculate
50 factorial: ");
51                 string? numInput = Console.ReadLine();
52                 if (int.TryParse(numInput, out number) && number >= 0)
53                     break;
```

```

53         Console.WriteLine("Invalid input. Please enter a non-
negative integer.");
54     }
55
56     long factorial = Factorial(number);
57     Console.WriteLine($"Factorial of {number} is {factorial}");
58 }
59 }
60 }

```

The output is shown below:



```

Microsoft Visual Studio Debug Console
For loop:
1 2 3 4 5 6 7 8 9 10

Foreach loop:
1 2 3 4 5 6 7 8 9 10

Enter something (type 'exit' to quit): Hellloo
You entered: Hellloo
Enter something (type 'exit' to quit): exit
Enter a non-negative integer to calculate factorial: 15
Factorial of 15 is 1307674368000

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\STT-CSE\LAB_9\STT_LAB_9\bin\Debug\net8.0\STT_LAB_9.exe (process 288)
exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 9.6: Output of program with for loops and factorial

9.3.3 Usage of single and multi-dimensional arrays

We now write a program to demonstrate the use of single and multi-dimensional arrays in C#. The program sorts an array of integers using the bubble sort algorithm without using built-in functions. It then shows how to store a 2D array into a 1D array in both row-major and column-major order. Finally, it performs matrix multiplication of two matrices and displays the result. This program helps in understanding basic array operations and how to work with them in C#.

```

1  using System;
2
3  namespace ArrayDemo
4  {
5      class ArrayOperations
6      {
7          public static void BubbleSort(int[] arr)
8          {
9              int n = arr.Length;
10             for (int i = 0; i < n - 1; i++)
11             {
12                 for (int j = 0; j < n - i - 1; j++)
13                 {
14                     if (arr[j] > arr[j + 1])
15                     {
16                         int temp = arr[j];
17                         arr[j] = arr[j + 1];
18                         arr[j + 1] = temp;
19                     }
20                 }
21             }
22         }
23     }

```



```
24 public static int[] FlattenRowMajor(int[,] matrix)
25 {
26     int rows = matrix.GetLength(0);
27     int cols = matrix.GetLength(1);
28     int[] result = new int[rows * cols];
29
30     int index = 0;
31     for (int i = 0; i < rows; i++)
32         for (int j = 0; j < cols; j++)
33             result[index++] = matrix[i, j];
34
35     return result;
36 }
37
38 public static int[] FlattenColumnMajor(int[,] matrix)
39 {
40     int rows = matrix.GetLength(0);
41     int cols = matrix.GetLength(1);
42     int[] result = new int[rows * cols];
43
44     int index = 0;
45     for (int j = 0; j < cols; j++)
46         for (int i = 0; i < rows; i++)
47             result[index++] = matrix[i, j];
48
49     return result;
50 }
51
52 public static int[,] MultiplyMatrices(int[,] A, int[,] B)
53 {
54     int rowsA = A.GetLength(0);
55     int colsA = A.GetLength(1);
56     int rowsB = B.GetLength(0);
57     int colsB = B.GetLength(1);
58
59     if (colsA != rowsB)
60         throw new InvalidOperationException("Number of columns in A
61 must equal number of rows in B.");
62
63     int[,] C = new int[rowsA, colsB];
64
65     for (int i = 0; i < rowsA; i++)
66     {
67         for (int j = 0; j < colsB; j++)
68         {
69             C[i, j] = 0;
70             for (int k = 0; k < colsA; k++)
71             {
72                 C[i, j] += A[i, k] * B[k, j];
73             }
74         }
75     }
76
77     return C;
78 }
79
80 public static void PrintArray(int[] arr)
81 {
82     foreach (int num in arr)
```

```
83         Console.Write(num + " ");
84     }
85     Console.WriteLine();
86 }
87
88 public static void PrintMatrix(int[,] matrix)
89 {
90     int rows = matrix.GetLength(0);
91     int cols = matrix.GetLength(1);
92     for (int i = 0; i < rows; i++)
93     {
94         for (int j = 0; j < cols; j++)
95         {
96             Console.Write(matrix[i, j] + "\t");
97         }
98         Console.WriteLine();
99     }
100 }
101
102
103 class Program
104 {
105     static void Main(string[] args)
106     {
107         int[] arr = { 5, 2, 9, 1, 5, 6 };
108         Console.WriteLine("Original Array:");
109         ArrayOperations.PrintArray(arr);
110
111         ArrayOperations.BubbleSort(arr);
112         Console.WriteLine("Sorted Array (Bubble Sort):");
113         ArrayOperations.PrintArray(arr);
114         Console.WriteLine();
115
116         int[,] matrix2D = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
117         Console.WriteLine("2-D Matrix:");
118         ArrayOperations.PrintMatrix(matrix2D);
119
120         int[] rowMajor = ArrayOperations.FlattenRowMajor(matrix2D);
121         Console.WriteLine("Flattened (Row-Major):");
122         ArrayOperations.PrintArray(rowMajor);
123
124         int[] colMajor = ArrayOperations.FlattenColumnMajor(matrix2D);
125         Console.WriteLine("Flattened (Column-Major):");
126         ArrayOperations.PrintArray(colMajor);
127         Console.WriteLine();
128
129         int[,] A = { { 1, 2 }, { 3, 4 } };
130         int[,] B = { { 5, 6 }, { 7, 8 } };
131         Console.WriteLine("Matrix A:");
132         ArrayOperations.PrintMatrix(A);
133         Console.WriteLine("Matrix B:");
134         ArrayOperations.PrintMatrix(B);
135
136         int[,] C = ArrayOperations.MultiplyMatrices(A, B);
137         Console.WriteLine("Matrix C = A x B:");
138         ArrayOperations.PrintMatrix(C);
139     }
140 }
141 }
```

The output is shown below:

```

Original Array:
5 2 9 1 5 6
Sorted Array (Bubble Sort):
1 2 5 5 6 9

2-D Matrix:
1      2      3
4      5      6
7      8      9
Flattened (Row-Major):
1 2 3 4 5 6 7 8 9
Flattened (Column-Major):
1 4 7 2 5 8 3 6 9

Matrix A:
1      2
3      4
Matrix B:
5      6
7      8
Matrix C = A x B:
19      22
43      50

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\STT-CSE\LAB_9\STT_LAB_9\bin\Debug\net8.
).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Au
Press any key to close this window . . .|

```

Figure 9.7: Output of program with bubblesort and matmul

9.4 Output Reasoning Analyses

9.4.1 Output Reasoning (Level 0)

We will now analyze the following C# programs and predict the outputs.

Program 1

```

1 using System; // namespace
2
3 class Program // default visibility is 'internal' if not specified
4 {
5     public static void Main(string[] args)
6     {
7         int a = 0; // default visibility is 'private' (in a class)
8         Console.WriteLine(a++);
9     }
10 }

```

Listing 9.1: Program 1

This piece of code should ideally print 0, as the ++ operator applied on *a* as a postoperator (*a*++) first returns *a* as it is (by making a copy of it), and then increments it by 1 in place.

Program 2

```

1 using System;
2 class Program
3 {
4     public void Main(string[] args)

```

```

5      {
6          int a = 0;
7          Console.WriteLine(a++);
8      }
9  }

```

Listing 9.2: Program 2

This program will not run because the Main method is not static.

When a C# program starts, the runtime looks for a static Main method as the entry point. Since Main here is defined as a regular (non-static) method, the program has no valid starting point and will show a compiler error:

"Program does not contain a static 'Main' method suitable for an entry point."

Here is the screenshot proving our point:

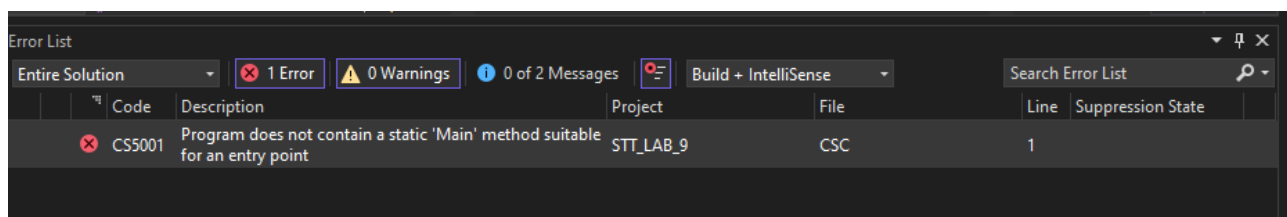


Figure 9.8: Build error of Program 2

9.4.2 Output Reasoning (Level 1)

Program 3

```

1  class Program
2  {
3      public static void Main(string[] args)
4      {
5          int a = 0;
6          int b = a++;
7          Console.WriteLine(a++.ToString(), ++a, -a++);
8          Console.WriteLine((a++).ToString() + (-a++).ToString());
9          Console.WriteLine(~b);
10     }
11 }

```

Listing 9.3: Program 3

Initially, $a = 0$. After line 6, $b = 0$ and $a = 1$. Now, the function signature of `Console.WriteLine` is `Console.WriteLine(string format, params object[] args)`. The first argument is treated as format string, containing placeholders like 0, 1, etc. So here, `a++.ToString()` outputs "1" and makes $a = 2$. Now, The other two arguments are 3 (with $a = 3$) and -3 (with $a = 4$). So effectively, the line becomes `Console.WriteLine("1", 3, -3)`. This just prints 1, as it does not contain any placeholders. Now, in line 8, the line becomes `Console.WriteLine("4-5"` with $a = 6$, and 4-5 gets printed in the next line. At line 9, `~b`, which is the bitwise NOT of b gets printed, which is `~0`, which is -1.

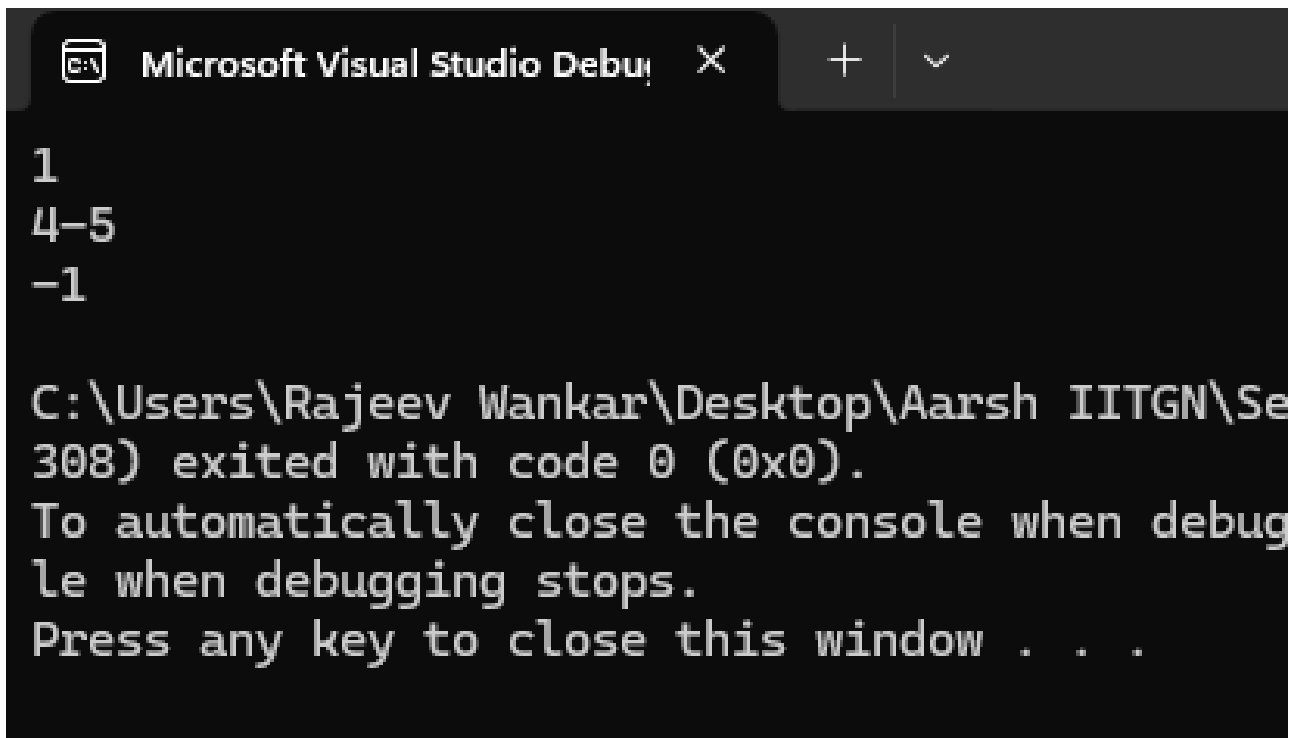
So the final expected output is:

```

1
4-5
-1

```

The screenshot of the output shown below conforms to our prediction.



```

1
4-5
-1

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Se
308) exited with code 0 (0x0).
To automatically close the console when debug
le when debugging stops.
Press any key to close this window . . .

```

Figure 9.9: Output of Program 3

Program 4

```

1 using System;
2 /*you can also write top level code outside of a class. C# takes
3 care of this by providing internal entry point Main*/
4 Console.WriteLine("int x = 3;");
5 Console.WriteLine("int y = 2 + ++x;");
6 int x = 3; //default visibility is internal (outside a class)
7 int y = 2 + ++x;
8 Console.WriteLine($"x = {x} and y = {y}");
9 Console.WriteLine("x = 3 << 2;");
10 Console.WriteLine("y = 10 >> 1;");
11 x = 3 << 2;
12 y = 10 >> 1;
13 Console.WriteLine($"x = {x} and y = {y}");
14 x = ~x;
15 y = ~y;
16 Console.WriteLine($"x = {x} and y = {y}");

```

Listing 9.4: Program 4

Initially, no variables are defined. The first two `Console.WriteLine` statements simply print the lines `int x = 3;` and `int y = 2 + ++x;` as text, they do not execute any code yet.

Now, the statement `int x = 3;` initializes `x` to 3, and `int y = 2 + ++x;` first increments `x` (so `x = 4`) and then computes `y = 2 + 4 = 6`. The next line uses string interpolation to print these values, giving the output `x = 4` and `y = 6`.

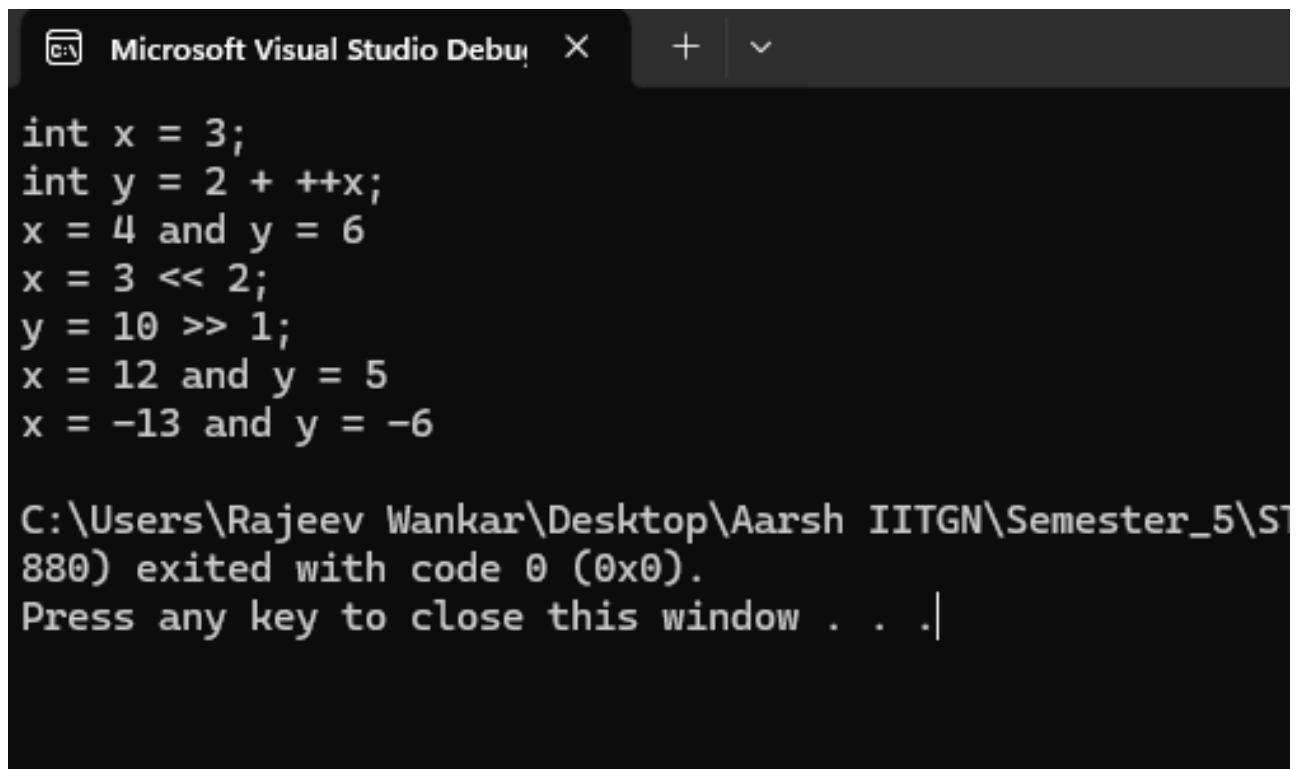
Next, the program prints two more lines of text: `x = 3 << 2;` and `y = 10 >> 1;` Again, these are just printed as strings.

After that, `x = 3 << 2;` performs a left shift operation on 3 by 2 bits, giving `x = 12`, and `y = 10 >> 1;` performs a right shift on 10 by 1 bit, giving `y = 5`. Thus, the next output line is `x = 12` and `y = 5`.

Finally, the statements `x = ~x;` and `y = ~y;` take the bitwise NOT of each variable. The bitwise NOT of 12 is -13, and that of 5 is -6, since $\sim n = -(n + 1)$ for integers. Therefore, the last line printed is `x = -13` and `y = -6`.

The final prediction for the output is:

```
int x = 3;
int y = 2 + ++x;
x = 4 and y = 6
x = 3 << 2;
y = 10 >> 1;
x = 12 and y = 5
x = -13 and y = -6
```



```
Microsoft Visual Studio Debug Console
int x = 3;
int y = 2 + ++x;
x = 4 and y = 6
x = 3 << 2;
y = 10 >> 1;
x = 12 and y = 5
x = -13 and y = -6

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\ST
880) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Figure 9.10: Output of Program 4

9.4.3 Output Reasoning (Level 2)

Program 5

```
1 using System;
2 public class Program
3 {
4     static void Main()
5     {
6         try
7         {
8             int i = int.MaxValue;
9             Console.WriteLine(-(i + 1) - i);
10            for (i = 0; i <= int.MaxValue; i++) ; //note semicolon here
11            Console.WriteLine("Program ended!");
12        }
13        catch (Exception ex)
14        {
15            Console.WriteLine(ex.ToString());
16        }
17    }
```

18 }

Listing 9.5: Program 5

Initially, the program sets `i` to `int.MaxValue`, which is the largest possible integer in C# (2,147,483,647). It then evaluates the expression `-(i + 1) - i`. When `i + 1` is computed, it overflows and wraps around to -2,147,483,648, since integer arithmetic in C# is unchecked by default. The expression becomes `-(-2,147,483,648) - 2,147,483,647`, which equals `2,147,483,648 - 2,147,483,647 = 1`. Therefore, the first line printed is 1. Next comes the for loop: `for (i = 0; i <= int.MaxValue; i++)`;

The semicolon at the end means the loop has no body, so it simply increments `i` repeatedly until the condition becomes false. However, when `i` reaches its maximum value (`int.MaxValue`), the next increment (`i++`) causes an overflow, wrapping `i` around to `int.MinValue` (-2,147,483,648).

At this point, the loop condition `i <= int.MaxValue` still holds true, because -2,147,483,648 is indeed less than 2,147,483,647. As a result, the loop never stops — it continues running indefinitely, and the line `Console.WriteLine("Program ended!");` is never reached.

Since no exceptions occur, the catch block is never executed.

So the final behavior of the program is that it prints:

1

and then runs forever without ever printing "Program ended!".

The following screenshot of the output verifies this.

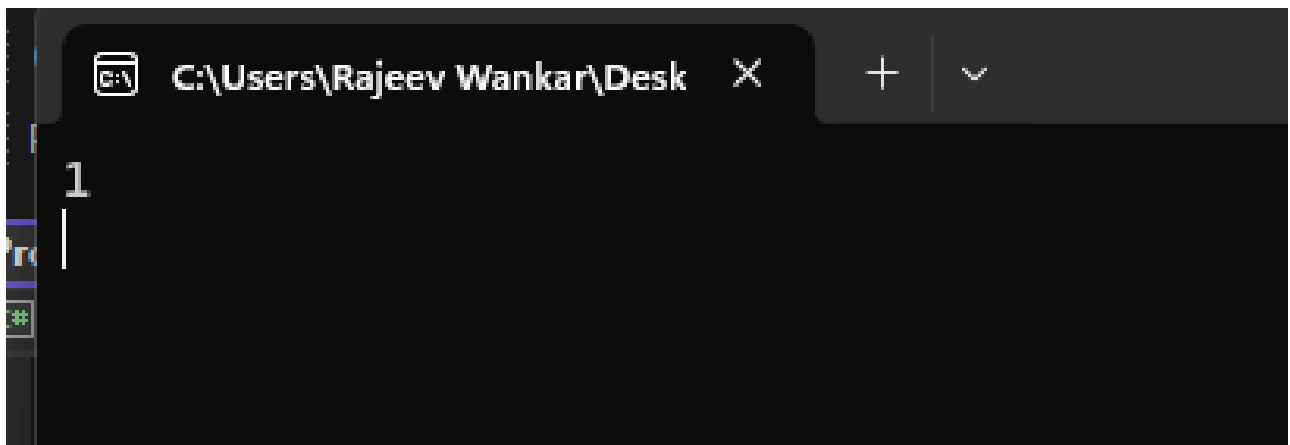


Figure 9.11: Output of program 11

Program 6

```

1 using System;
2 public class Program
3 {
4     static void Main(string[] args)
5     {
6         Main(["CS202"]);
7     }
8 }

```

Listing 9.6: Program 5

The program defines a `Main` method that takes a string array as an argument. Inside it, the same `Main` method is called again with `["CS202"]` as input. The syntax `["CS202"]` is valid in newer versions of C# (C# 12 and above) and creates a string array with one element "CS202".

Since the program calls `Main` inside itself with no stopping condition, it keeps calling itself infinitely. Each new call is added to the call stack, and after many recursive calls, the stack memory becomes full. This causes the program to crash with a `StackOverflowException`.

Here is the screenshot of the Exception:

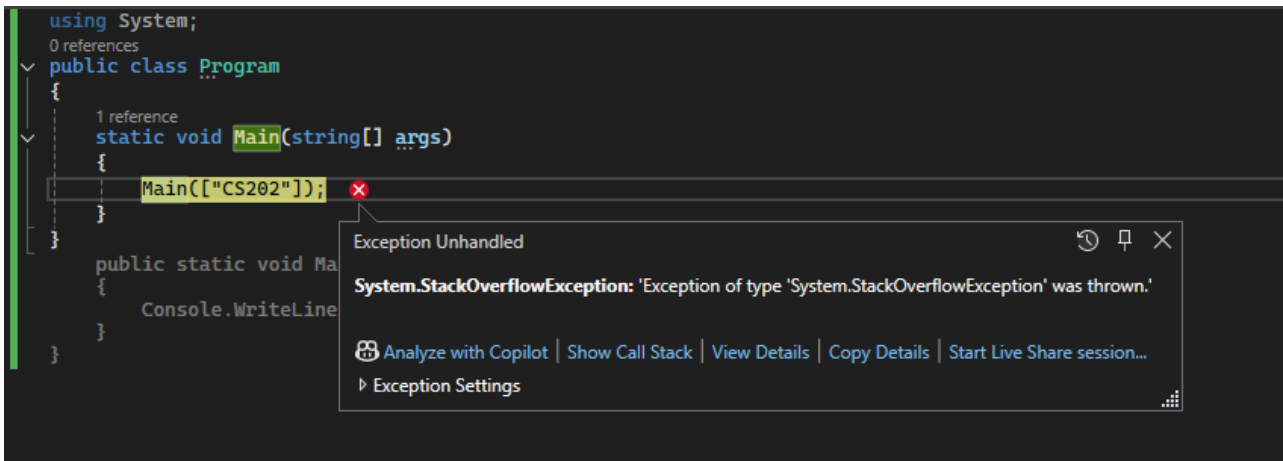


Figure 9.12: Program 6 Stack Overflow

9.5 Conclusion

This lab increased our understanding of the concepts of C# programming and the process of developing and executing console applications within the .NET framework. We wrote and tested multiple programs, which helped us to know about how various control structures, loops, and functions operate in C#.

References

- [1] "Lab assignment document," Google Classroom, accessed: 2025-11-05.
- [2] M. Corporation, "C# guide - .net managed language," <https://learn.microsoft.com/en-us/dotnet/csharp/>, 2025, accessed: 2025-11-08.

Lab 10

Object Lifecycle, Inheritance, and Event-Driven Programming in C#

[Github Repository for Lab 10](#)

10.1 Introduction

In this lab, we practice object-oriented programming in C# by building small console programs. We create classes with constructors and destructors, use inheritance to share and override behaviors, and work with events and delegates to make programs respond to actions. The goal is to learn how to write organized, reusable code that models real-world situations.

10.2 Setup and Tools

We use Visual Studio 2022 Community Version for .NET development and writing C# code along with .NET 8.0 (LTS) SDK. We use Windows 11 as our operating system. We have already set up the .NET Development Environment, in the previous lab.

10.3 Methodology, Execution and Results

10.3.1 Constructors and Data Control

We write a program which defines a class named Program containing a private integer field called data. The class includes a constructor that initializes the object and prints the message "Constructor Called", and a destructor that displays "Object Destroyed" when the object is cleaned up. Two member methods are implemented: `set_data(int x)`, which assigns a value to the data field, and `show_data()`, which prints the current value of data to the console.

In the `Main()` method, three objects of the Program class are dynamically created. The `set_data()` method is used to assign the values 10, 20, and 30 to these objects respectively, and their corresponding values are displayed sequentially using the `show_data()` method.

To observe object lifecycle behavior, a static counter variable is added to the class to track the number of currently active objects. This counter is incremented within the constructor and decremented within the destructor, and its value is displayed each time an object is created or destroyed. Upon program termination, the destructor messages are observed to understand when each object is actually destroyed by the garbage collector.

The code is shown below:

```
1 using System;
2 namespace Program
3 {
4     class Program
```

```

5      {
6          private int data;
7          static int counter = 0;
8          public Program()
9          {
10             Console.WriteLine("Constructor called!");
11             counter++;
12             Console.WriteLine($"Counter value: {counter}");
13         }
14         ~Program()
15         {
16             Console.WriteLine("Destructor called, object cleaned up!");
17             counter--;
18             Console.WriteLine($"Counter value: {counter}");
19         }
20         public void set_data(int value)
21         {
22             data = value;
23         }
24         public void show_data()
25         {
26             Console.WriteLine(data);
27         }
28     }
29     class Test
30     {
31         static void runner()
32         {
33             Program obj1 = new Program();
34             Program obj2 = new Program();
35             Program obj3 = new Program();
36
37             obj1.set_data(10);
38             obj2.set_data(20);
39             obj3.set_data(30);
40
41             obj1.show_data();
42             obj2.show_data();
43             obj3.show_data();
44         }
45         static void Main(string[] args)
46         {
47             runner();
48             GC.Collect();           // Force garbage collection
49             GC.WaitForPendingFinalizers(); // Wait for destructors to run
50         }
51     }
52 }

```

The output is as follows:

```

Constructor called!
Counter value: 1
Constructor called!
Counter value: 2
Constructor called!
Counter value: 3
10
20
30
Destructor called, object cleaned up!
Counter value: 2
Destructor called, object cleaned up!
Counter value: 1
Destructor called, object cleaned up!
Counter value: 0

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\STT-CSE\LAB
460) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 10.1: Output of the constructor destructor program

In C#, the execution of destructors is non-deterministic because object destruction is managed automatically by the garbage collector rather than occurring at a specific point in the program. When an object goes out of scope, its memory is not immediately reclaimed; instead, it becomes eligible for garbage collection. The garbage collector runs only when the runtime determines that memory recovery is necessary, which means the exact moment when a destructor is executed cannot be predicted. As a result, destructor messages may not appear immediately after an object is no longer referenced. To ensure that destructors are executed within this program, the statements `GC.Collect()` and `GC.WaitForPendingFinalizers()` are explicitly used to manually trigger garbage collection and wait for all pending finalizers to complete. This approach is used only for demonstration purposes, as forcing garbage collection is generally discouraged in production code.

A separate `runner()` function is defined to localize object creation and ensure that all objects go out of scope before garbage collection is explicitly invoked. When `runner()` finishes execution, the objects created within it (`obj1`, `obj2`, and `obj3`) are no longer accessible, making them immediately eligible for garbage collection. If the objects were created directly inside the `Main()` method, they would remain in scope until the end of `Main()`, and the garbage collector would not destroy them until after the program terminates. By isolating object creation in `runner()`, the program ensures that object destruction can be observed in a controlled and predictable manner when the garbage collector is explicitly triggered.

10.3.2 Inheritance and Method Overriding

In this question, we create a program to demonstrate inheritance and method overriding in C#. We define a base class `Vehicle` with protected fields `speed` and `fuel`, and two virtual methods: `Drive()` and `ShowInfo()`. Then, we create two derived classes, `Car` and `Truck`, that each add their own field and override these methods to show different messages and change the fuel value differently. In the `Main()` method, we create one object of each class and store them in a `Vehicle[]` array. By looping through the array and calling the `Drive()` and `ShowInfo()` methods using base-class references, we can see how each object behaves differently. This shows how runtime polymorphism allows base-class references to call the correct overridden methods from the derived classes.

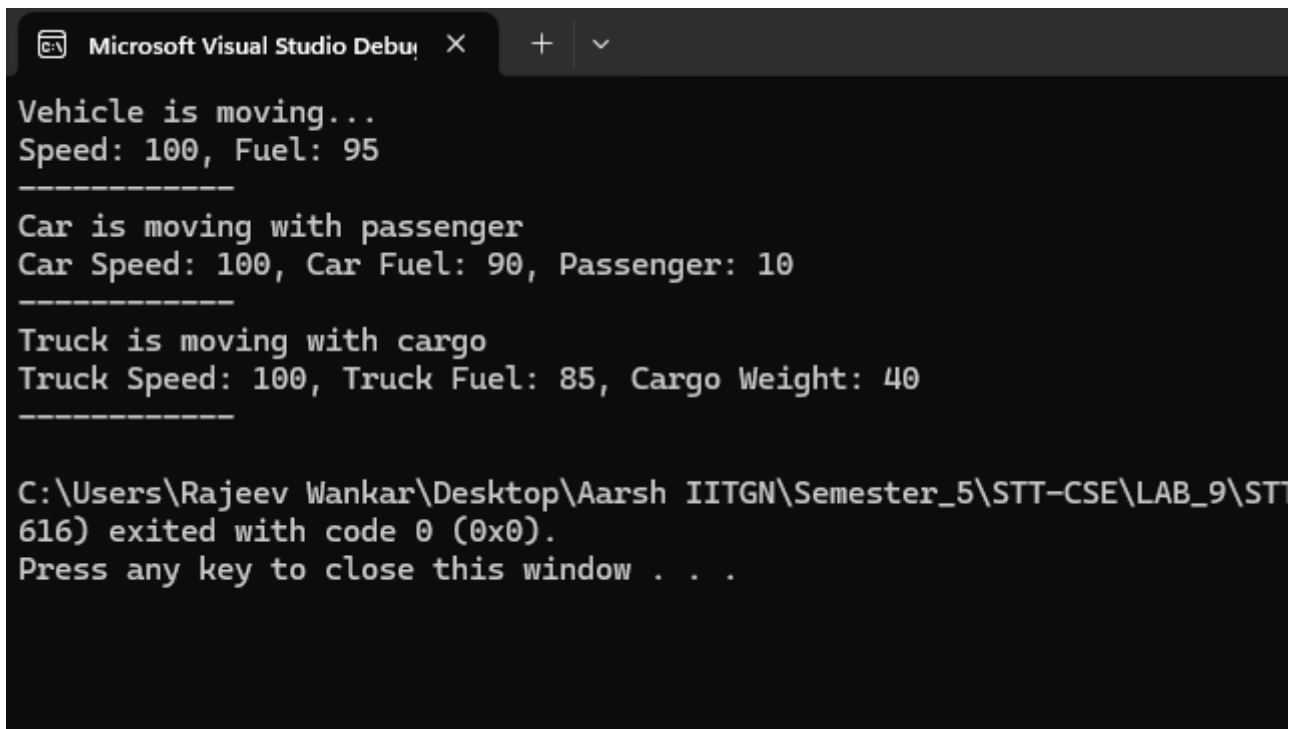
The code can be found here:

```

1  using System;
2
3  namespace Program
4  {
5      class Vehicle
6      {
7          protected int speed = 100;
8          protected int fuel = 100;
9          public virtual void ShowInfo() => Console.WriteLine($"Speed: {speed
10 }, Fuel: {fuel}");
11          public virtual void Drive()
12          {
13              Console.WriteLine($"Vehicle is moving...");
14              fuel -= 5;
15          }
16      }
17
18      class Car : Vehicle
19      {
20          public int passengers = 10;
21
22          public override void ShowInfo() => Console.WriteLine($"Car Speed: {
23 speed}, Car Fuel: {fuel}, Passenger: {passengers}");
24          public override void Drive()
25          {
26              Console.WriteLine("Car is moving with passenger");
27              fuel -= 10;
28          }
29      }
30
31      class Truck : Vehicle
32      {
33          public int cargoWeight = 40;
34          public override void ShowInfo() => Console.WriteLine($"Truck Speed:
35 {speed}, Truck Fuel: {fuel}, Cargo Weight: {cargoWeight}");
36          public override void Drive()
37          {
38              Console.WriteLine("Truck is moving with cargo");
39              fuel -= 15;
40          }
41      }
42
43      class Program
44      {
45          static void Main(string[] args)
46          {
47              Vehicle[] vehicles = { new Vehicle(), new Car(), new Truck() };
48              for (int i = 0; i < vehicles.Length; i++)
49              {
50                  vehicles[i].Drive();
51                  vehicles[i].ShowInfo();
52                  Console.WriteLine("-----");
53              }
54          }
55      }
56  }

```

The output is as follows:



```

Microsoft Visual Studio Debug Console
+ -

Vehicle is moving...
Speed: 100, Fuel: 95
-----
Car is moving with passenger
Car Speed: 100, Car Fuel: 90, Passenger: 10
-----
Truck is moving with cargo
Truck Speed: 100, Truck Fuel: 85, Cargo Weight: 40
-----

C:\Users\Rajeev Wankar\Desktop\Aarsh IITGN\Semester_5\STT-CSE\LAB_9\STT
616) exited with code 0 (0x0).
Press any key to close this window . . .

```

Figure 10.2: Output of program showing runtime polymorphism

In this program, the `Vehicle` class defines two virtual methods: `Drive()` and `ShowInfo()`. The derived classes `Car` and `Truck` both override these methods to change their behavior.

When we store objects of `Vehicle`, `Car`, and `Truck` in a `Vehicle[]` array and call `Drive()` and `ShowInfo()` through a base-class reference, the overridden methods in the derived classes are executed.

This happens because C# uses runtime polymorphism — the actual method that gets called depends on the runtime type of the object, not the compile-time type of the reference.

- For the `Vehicle` object, the base methods are called.
- For the `Car` object, the overridden methods in `Car` are executed.
- For the `Truck` object, the overridden methods in `Truck` are executed.

This demonstrates that overriding changes method behavior by allowing derived classes to replace base class functionality. The base-class references (`Vehicle`) can therefore point to different object types and automatically call the correct method version at runtime via the virtual method table (vtable) mechanism managed by the (Common Language Runtime) CLR.

10.4 Output Reasoning Analyses

10.4.1 Output Reasoning (Level 0)

Program 1

```

1 using System;
2 int a = 3;
3 int b = a++;
4 Console.WriteLine($"a is {a++}, b is {--b}");
5 int c = 3;
6 int d = ++c;
7 Console.WriteLine($"c is {-c--}, d is {~d}");

```

Listing 10.1: Program 1

The output will be as follows:

```
a is 4, b is -4
c is -4, d is -5
```

The code initializes $a = 3$ and assigns $b = a++$, which means b gets the original value of a (3), and then a is incremented to 4. The `Console.WriteLine` prints a is 4, as $a++$ increments a after its value is used, and b is -4, as $++b$ first increments b to 4, and then negates it. Next, c is set to 3 and d to $++c$, so c is incremented to 4 before being assigned to d , making both c and d equal to 4. The second `Console.WriteLine` prints c is -4, as $-c$ first negates the current value of c (4), and then decrements c to 3 after its value is used, and d is -5, as $\sim d$ applies the bitwise NOT to d (4), resulting in -5.

Program 2

```
1 using System;
2
3 class Program
4 {
5     int age;
6
7     Program() => age = age == 0 ? age + 1 : age - 1;
8
9     static void Main()
10    {
11        int k = "010%".Replace('0', '%').Length;
12
13        Console.Write("[ " + (k << ++new Program().age).ToString() + " ]");
14        Console.Write("[ " + "010%".Split('1')[1][0] + " ]");
15        Console.Write("[ " + "010%".Split('0')[1][0] + " ]");
16        Console.Write("[ " + int.Parse(Convert.ToString("123".ToCharArray()
17        [~-1])) + " ]");
18    }
```

Listing 10.2: Program 2

The output will be:

```
[16] [0] [1] [1]
```

The program performs a series of string manipulations and bitwise operations:

1. It replaces '0' with '%' in '010%', resulting in the string '%%1%', and then calculates its length, which is '4'. So, value of k is 4.
2. It creates an object of 'Program', which increments 'age' to '2'. The left-shift operation ($4 \ll 2$) results in '16'.
3. It splits the string '010%' by '1', takes the second part '0%', and extracts the first character '0'.
4. It splits '010%' by '0', takes the second part '1%', and extracts the first character '1'.
5. It converts the first character of '123', which is '1', to an integer, resulting in '1'.

Program 3

```
1 using System;
2
3 class Program
4 {
```

```

5      static void Main()
6      {
7          int[] nums = { 0, 1, 0, 3, 12 };
8          int pos = 0;
9
10         for (int i = 0; i < nums.Length; i++)
11         {
12             if (nums[i] != 0)
13             {
14                 int temp = nums[pos];
15                 nums[pos] = nums[i];
16                 nums[i] = temp;
17                 pos++;
18             }
19         }
20
21         Console.WriteLine(string.Join(", ", nums));
22     }
23 }

```

Listing 10.3: Program 3

The output will be as follows:

1, 3, 12, 0, 0

This is because the given code effectively pushes the zeros of the array to the right end while keeping the order of the remaining elements the same.

10.4.2 Output Reasoning (Level 1)

Program 4

```

1  using System;
2
3  class Program
4  {
5      int age;
6
7      Program() => age = age == 0 ? age + 1 : age - 1;
8
9      static void Main()
10     {
11         int k = "010%".Replace('0', '%').Length;
12
13         Console.Write "[" + (k << ++new Program().age).ToString() + "]");
14         Console.Write "[" + "010%".Split('1')[1][0] + "]");
15         Console.Write "[" + "010%".Split('0')[1][0] + "]");
16         Console.Write "[" + int.Parse(Convert.ToString("123".ToCharArray()
17         [~-1])) + "]");
18     }
19 }

```

Listing 10.4: Program 4

This is same as **Program 2**, hence, the output will be

[16] [0] [1] [1]

Program 5

```

1 using System;
2
3 class Program
4 {
5     int f;
6
7     public static void Main(string[] args)
8     {
9         Console.WriteLine("run 1");
10        Program p = new Program(new int() + "0".Length);
11
12        for (int i = 0, _ = i; i < 5 && ++p.f >= 0; i++, Console.WriteLine(p
13            .f++));
14
15        {
16            for (; p.f == 0; );
17
18            {
19                Console.WriteLine(p.f);
20            }
21
22            Console.WriteLine("\nrun 2");
23            p = new Program(p.f);
24            Console.WriteLine(p.f);
25
26            Console.WriteLine("\nrun 3");
27            p = new Program();
28            Console.WriteLine(p.f);
29        }
30
31        Program() => f = 0;
32        Program(int x) => f = x;
33    }

```

Listing 10.5: Program 5

Firstly it prints 'run 1'. See that, the first for loop on line 12 has no body (notice the semicolon ; at the end). Initially, `p.f` is set to 1 from the constructor call:

$$\text{new Program}(\text{new int}() + \text{"0"}.Length) \Rightarrow 0 + 1 = 1.$$

In each iteration of the loop:

- The condition `++p.f >= 0` pre-increments `p.f`.
- The statement `Console.WriteLine(p.f++)` prints the current value and then post-increments `p.f`.

The value of `p.f` changes as follows:

Iteration	Before Condition	After ++p.f	Printed Value	After p.f++
1	1	2	2	3
2	3	4	4	5
3	5	6	6	7
4	7	8	8	9
5	9	10	10	11

After the fifth iteration, `i = 5`, so the loop terminates with `p.f = 11`.

The next block is independent of the first loop. Inside it, there is another for loop:

```
for (; p.f == 0; ) ;
```

Since `p.f = 11`, the condition is false, and this loop never executes. The subsequent statement `Console.WriteLine(p.f)` simply prints the current value, which is **11**.

It now prints 'run 2' and, a new `Program` object is created with `p.f = 11`, so the constructor sets `f = 11`, which is printed.

It prints 'run 3' and, the parameterless constructor `Program()` sets `f = 0`, so the output is **0**.

The output will be:

```
run 1
2
4
6
8
10
11

run 2
11

run 3
0
```

Program 6

```

1 public class A
2 {
3     public virtual void f1()
4     {
5         Console.WriteLine("f1");
6     }
7 }
8
9
10 public class B : A
11 {
12     public override void f1() => Console.WriteLine("f2");
13 }
14
15 class Program
16 {
17     static int i = 0;
18     public event funcPtr handler;
19     public delegate void funcPtr();
20
21     public void destroy()
22     {
23         if (i == 6)
24         {
25             return;
26         }
27         else
28         {
29             Console.WriteLine(i++);
30             destroy();
31         }

```

```

32     }
33
34     public static void Main(string[] args)
35     {
36         Program p = new Program();
37         p.handler += new funcPtr((new A()).f1);
38         p.handler += new funcPtr((new B()).f1);
39         p.handler();
40         p.handler -= new funcPtr((new B()).f1);
41         p.handler -= new funcPtr((new A()).f1);
42         p?.destroy(); //check here about ?. operator
43         p = null;
44         i = -6;
45         p?.destroy();
46         (new Program())?.destroy();
47     }
48 }

```

Listing 10.6: Program 6

The output of this is:

```

f1
f2
0
1
2
3
4
5
-6
-5
-4
-3
-2
-1
0
1
2
3
4
5

```

The program's output is generated in four distinct phases:

1. Event Invocation The handler event is assigned methods from new A and B instances. A.f1 prints "f1", and the overridden B.f1 prints "f2". Calling handler() executes both subscribed methods in order.

- **Output:** f1, f2

2. First Recursive destroy() Call The call p?.destroy() executes because p is not null. The destroy() method recursively prints the **static** variable i (starting at 0) and increments it. The recursion stops when i reaches 6.

- **Output:** 0, 1, 2, 3, 4, 5

- **Final i:** 6

3. State Change and Skipped Call First, `p` is set to `null`, and the **static** `i` is reset to `-6`. The expression `p?.destroy()` is then evaluated. The null-conditional operator (`?.`) detects that `p` is null and skips the method call entirely.

- **Output:** (None)

4. Second Recursive `destroy()` Call A new, non-null object is created via `(new Program())`, so `destroy()` is called. This method accesses the same **static** `i` (which is now `-6`) and recursively prints/increments it until `i` once again reaches `6`.

- **Output:** `-6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5`

About the `?` operator: The `?.` operator in C# is the null-conditional operator, designed specifically to prevent `NullReferenceException` errors by safely accessing members of potentially null objects. Essentially it performs this check: if the object on the left side of `?.` (like `p` in `p?.destroy()`) is not null, the member access or method call executes normally. However, if the object is null, the expression immediately short-circuits, skipping the member access entirely and preventing an error, thereby generating no output and simplifying code that would otherwise require verbose `if (p != null)` checks.

10.4.3 Output Reasoning (Level 2)

Program 7

```

1 using System;
2
3 public class Institute
4 {
5     internal int i = 7;
6
7     public Institute()
8     {
9         Console.Write("1");
10    }
11
12    public virtual void info()
13    {
14        Console.Write("2");
15    }
16 }
17
18 public class IITGN : Institute
19 {
20     public int i = 8;
21
22     public IITGN()
23     {
24         Console.Write("3");
25     }
26
27     public IITGN(int i)
28     {
29         Console.Write("4");
30     }
31
32     public override void info()
33     {
34         Console.Write("5");
35     }
36 }
37

```

```

38 class Program
39 {
40     public static void Main(string[] args)
41     {
42         Console.Write("6");
43
44         Institute ins1 = new Institute();
45         ins1.info();
46
47         IITGN ab101 = new IITGN(3);
48         ab101 = new IITGN();
49         ab101.info();
50
51         Console.WriteLine();
52         Console.WriteLine(ab101.i);
53         Console.WriteLine(~(((Institute)ab101).i));
54     }
55 }

```

Listing 10.7: Program 7

The expected output is:

```

61214135
8
-8

```

The program's execution begins with `Main` (printing "6"), followed by the `Institute` constructor ("1") and its `info` method ("2"). Next, an `IITGN` object is initialized using its parameterized constructor, which first invokes the base `Institute` constructor ("1") and then its own ("4"). This object is immediately replaced by a new `IITGN` instance using the default constructor, again invoking the base ("1") and then the derived ("3") constructors. A call to `ab101.info()` resolves polymorphically to the overridden `IITGN.info` method ("5"), completing the first line of output (61214135). Subsequent `WriteLine` calls demonstrate field hiding: `ab101.i` accesses the `IITGN` field (printing "8"), while `((Institute)ab101).i` accesses the hidden `Institute` field ("7"). The final output applies a bitwise NOT operator to the base field (`~7`), resulting in "-8".

Program 8

```

1
2 using System;
3
4 public class Program
5 {
6     public delegate void mydel();
7
8     public void fun1()
9     {
10         Console.WriteLine("fun1()");
11     }
12
13     public void fun2()
14     {
15         Console.WriteLine("fun2()");
16     }
17
18     public static void Main(string[] args)
19     {
20         Program p = new Program();
21     }

```

```

22     mydel obj1 = new mydel(p.fun1);
23     obj1 += new mydel(p.fun2);
24
25     Console.WriteLine("run 1");
26     obj1();
27
28     mydel obj2 = new mydel(p.fun2);
29     obj2 += new mydel(p.fun1);
30
31     Console.WriteLine("run 2");
32     obj2();
33
34     obj2 -= p.fun2;
35
36     Console.WriteLine("run 3");
37     obj2();
38 }
39 }

```

Listing 10.8: Program 8

The output will be:

```

run1
fun1()
fun2()
run2
fun2()
fun1()
run3
fun1()

```

This program shows how delegates can hold a list of multiple functions. First, the delegate `obj1` is set up to call `p.fun1`, and then `p.fun2` is added to its list with `+=`. When "run 1" calls `obj1()`, it runs both functions in that order, printing "fun1()" then "fun2()". Next, `obj2` is created, starting with `p.fun2`, and then `p.fun1` is added, making a *different* call list. The call after "run 2" runs them in this new order, printing "fun2()" then "fun1()". Finally, `p.fun2` is removed from `obj2`'s list using `-=`. The call after "run 3" now only runs the function that's left, `p.fun1`, printing just "fun1()".

Program 9

```

1
2 using System;
3 using System.Collections;
4
5 public class Program
6 {
7     int x;
8
9     public static void Main(string[] args)
10    {
11        ArrayList L = new ArrayList();
12
13        L.Add(new Program());
14        L.Add(new Program());
15
16        for (int i = 0; i < L.Count; i++)
17        {
18            Console.WriteLine(++((Program)L[i]).x);

```

```
19     }
20
21     L[0] = L[1];
22
23     ((Program)L[0]).x = 202;
24
25     for (int i = 0; i < L.Count; i++)
26     {
27         Console.WriteLine(((Program)L[i]).x);
28     }
29
30     ((Program)L[0]).x = 111;
31
32     L.RemoveAt(0);
33
34     Console.WriteLine(L.Count);
35     Console.WriteLine(((Program)L[0]).x);
36 }
37 }
```

Listing 10.9: Program 9

The output will be:

```
1
1
202
202
1
111
```

This program shows how an `ArrayList` handles object references. First, two new `Program` objects (let's call them P1 and P2) are added to the list, both with their `x` field at 0. The first loop goes through the list, increments the `x` for each (from 0 to 1), and prints "1" and "1". Then, the line `L[0] = L[1]` makes the first slot in the list point to the same object as the second slot (P2). Now the list is effectively [P2, P2]. When `x` in `L[0]` is set to 202, `x` in `L[1]` also becomes 202 because they are the same object, so the next loop prints "202" twice. `x` is then set to 111 (again, affecting both slots). When `L.RemoveAt(0)` is called, the first reference to P2 is removed, and the second reference to P2 slides down to index 0. The program then prints the new list count ("1") and the `x` value of the only object left ("111").

10.5 Conclusion

In this lab, we showed object-oriented programming principles in C#, including run-time polymorphism, as well as advanced features such as delegates and events. We also analyzed and reasoned through the behavior and output of various given programs.

References

- [1] Microsoft, "Constructors (c# programming guide)," <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constructors>, 2025, accessed: 2025-11-05.
- [2] "Lab assignment document," Google Classroom, accessed: 2025-11-05.
- [3] Microsoft, "Publisher-subscriber pattern (azure architecture center)," <https://learn.microsoft.com/en-us/azure/architecture/patterns/publisher-subscriber>, 2025, accessed: 2025-11-05.
- [4] —, "Tutorial: Inheritance – c# fundamentals," <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/inheritance>, 2025, accessed: 2025-11-05.